

Chapter-1

Introduction

1. Introduction

- **Objectives**

- To learn the basis of OO Analysis and design
- To have clear idea about traditional and modern SW development Methodologies and OOPS concepts.
- To identify objects, relationships, services and attributes
- To introduce the concept of Object-oriented system development lifecycle
- Discuss the overview of Object oriented methodologies

1.1 What is Object-oriented analysis and design

Defn

Object-oriented analysis and design (OOAD) is a popular technical approach for analyzing, designing an application, system, or business by applying the object oriented paradigm and visual modeling throughout the development life cycles for better communication and product quality.

OOA: we find and describe business objects or concepts in the problem domain

OOD: we define how these software objects collaborate to meet the requirements, Attributes and methods.

1.2 Software development

- Software development is dynamic and always undergoing major change.
- Systems development refers to all activities that go into producing an information systems solution.
- Systems development activities consist of systems analysis, modeling, design, implementation, testing and maintenance.

A software development methodology is a series of processes describe how the work is to be carried out to achieve the original goal based on the system requirements.

1.3 TWO ORTHOGONAL VIEWS OF THE SOFTWARE

Two Approaches,

- ✓ Traditional Approach
- ✓ Objected-Oriented Approach

Traditional Approach

Here Algorithms + Data structures = Programs. “A software system is a set of mechanisms for performing certain actions on certain data.”

Object-Oriented development offers a different model from the traditional software development approach, which is based on functions and procedures.

Difference between Traditional and Object Oriented Approach

Traditional Approach	Object oriented Approach
Collection of procedures	Combination of data and functions
Focuses on function and procedures different styles and methodologies for each step of process	Focuses on object, classes, modules that can be easily replaced, modifies and reused
Moving from one phase to another phase is complex	Moving from one phase to another phase is easier
Increases complexity	Reduce complexity and redundancy
Increases duration of the project	Decreases duration of the project

Object oriented systems development methodology

- **Object-Oriented development** offers a different model from the traditional software development approach, which is based on functions and procedures.
- In an object-Oriented system, everything is an object and each object is responsible for itself. A windows object is responsible for things like opening, sizing, and closing itself.

Why an Object Orientation?

To create sets of objects that work together concurrently to produce s/w that better, model their problem domain that similarly system produced by traditional techniques.

- It adapts to

1. Changing requirements
2. Easier to maintain
3. More robust
4. Promote greater design
5. Code reuse

Why an Object Orientation?

- The systems are easier to adapt to changing requirements, easier to maintain, more robust, and promote greater design and code reuse. Object-oriented development allows us to create module of functionality.
- Here are some reasons why object-Orientation works:
 - ✓ *Higher level of abstraction*
 - ✓ *Seamless transition among different phases of software development.*
 - ✓ *Encouragement of good programming techniques.*
 - ✓ *Promotion of reusability*

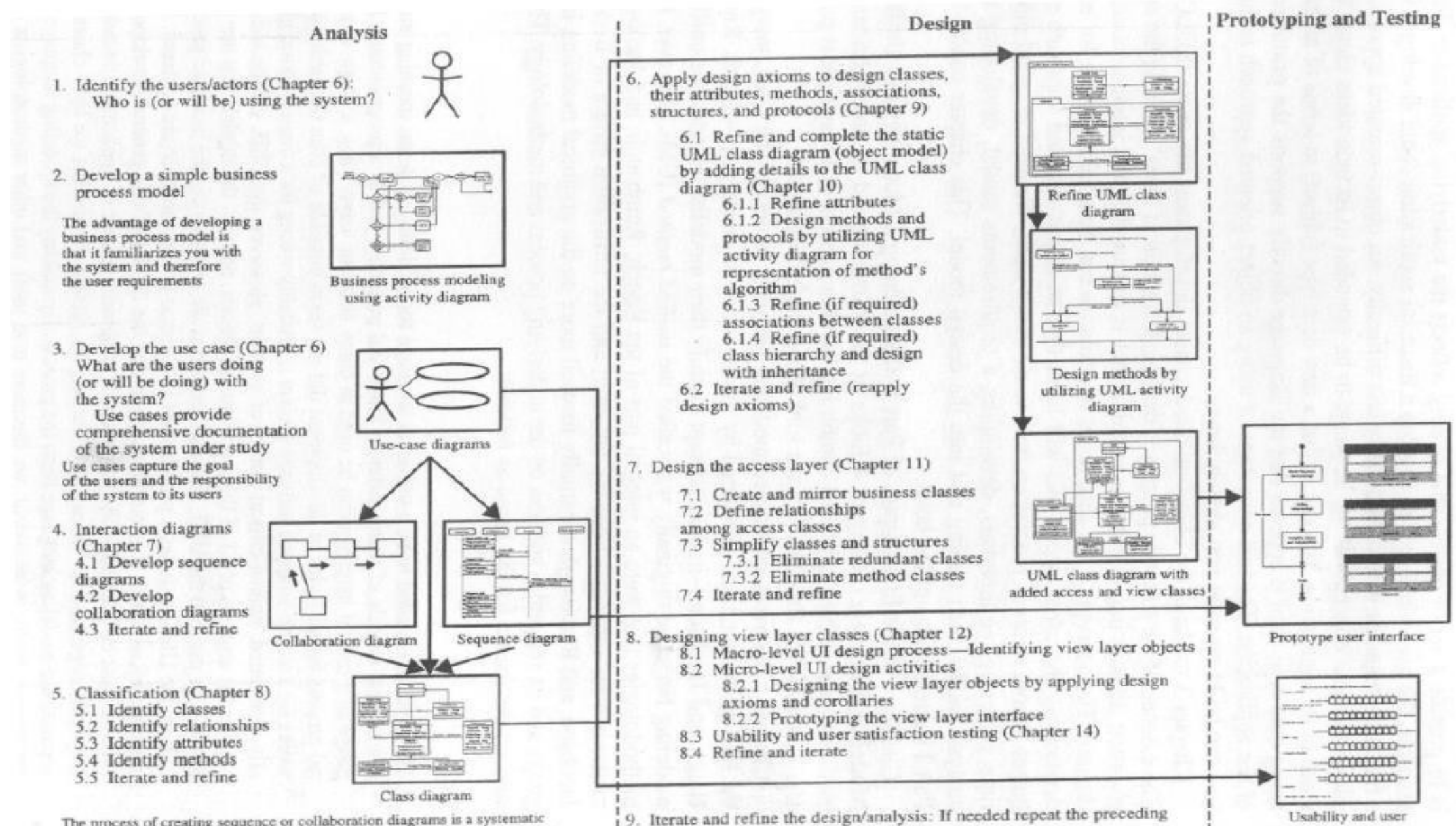
OVERVIEW OF THE UNIFIED APPROACH

- The *unified approach* (UA) is a methodology for software development that is proposed by the author.
- The UA, based on methodologies by Booch, Rumbaugh, and Jacobson, tries to combine the best practices, processes, and guidelines along with the Object Management Group's unified modeling language.

OVERVIEW OF THE UNIFIED APPROACH

- UML is a set of notations and conventions used to describe and model an application.
- UA utilizes the unified modeling language (UML) which is a set of notations and conventions used to describe and model an application.

The use case represents a typical interaction between a user and a computer system to capture the users' goals and needs.



The unified approach road map

- The main advantage of an object-oriented system is that the class tree is dynamic and can grow.
- Your function as a developer in an object-oriented environment is to foster the growth of the class tree by defining new, more specialized classes to perform the tasks your applications require.
- After your first few projects, you will accumulate a repository or class library of your own, one that performs the operations your applications most often require.
- At that point, creating additional applications will require no more than assembling classes from the class library.

Object Basics

Objects:

- The term **Object** means a combination of data and logic that represents some real world entity.
- ✓ When developing an object – oriented application, two basic questions always arise.
- What objects does the application need?
- What functionality should those objects have?

Object Basics

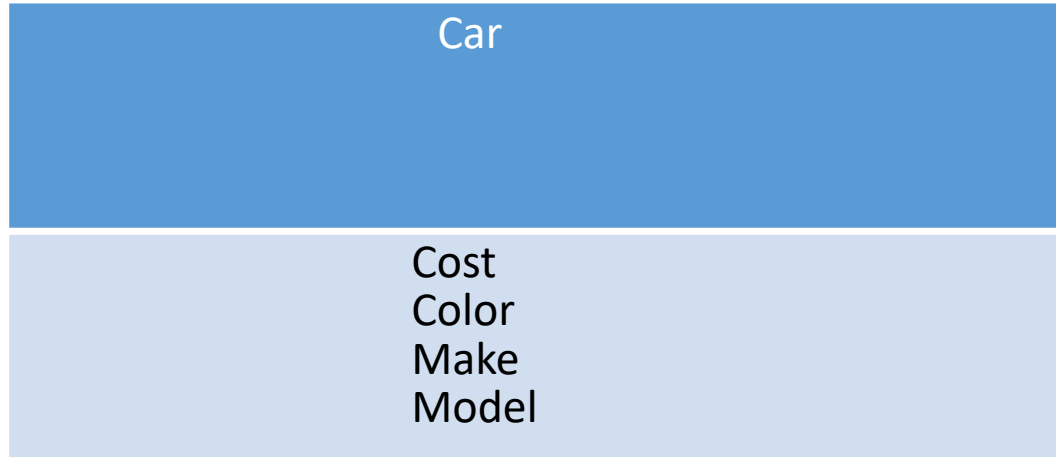
OBJECTS ARE GROUPED IN CLASSES

- A class is a specification of structures (instance variables), behavior (methods), and inheritance for objects.
- Classes are important mechanisms for classifying objects. In an object- oriented system, a *method* or behavior of an object is defined by its class. Each object is an instance of a class.
- The role of a class is to define the attributes and methods (the state and behavior) and applicability of its instances.
- **Object state and properties (attributes):-**

Properties represent the state of an object.

Example: the properties of a car, such as color, manufacturer and cost are abstract descriptions.

Object Basics



The attributes of a Car object.

For color, we could choose to use a sequence of characters such as red, etc.

The term object was first formally utilized in the Similar language to simulate some aspect of reality.

- An object is an entity.
 - It knows things (has attributes)
 - It does things (provides services or has methods)

Object's examples

Examples

It Knows things (attributes)

- I am an Students.
- I know my name,
- my ID number and
- my address.

It does things (methods)

- I know how to
- compute
- my payroll.
- Attributes or properties describe object's state (data) and methods define its behavior.

• *Attributes*

- I am a Car.
- I know my color,
- manufacturer, cost,
- owner and model.

Behavior (Object Behavior and methods)

- Behavior denotes the collection of methods that abstractly describes what an object is capable of doing.
- Each procedure defines and describes a particular behavior of an object.
- The object , called the receiver, is that on which the method operates.
- Methods encapsulate the behavior of the object, provide interface to the object, and hide any of the internal structures and states maintained by the object.
- Procedure provide us the means to communicate with an object and access its properties.
- Objects take responsibility for their own behavior.

Behavior (Object Behavior and methods)

We can drive a car, we can ride an elephant, or the elephant can eat a peanut. Each of these statements is a description of the object's behavior. In the object model, object behavior is described in methods or procedures. A method implements the behavior of an object.

Messages (Objects respond to messages)

- An object's capabilities are determined by the methods defined for it. Methods conceptually are equivalent to the functions definitions used in procedural languages. Objects perform operations in response to messages. For example, when you press on the brake pedal of a car, you send a stop message to the car object.

Behavior (Object Behavior and methods)

- **Messages** essentially are nonspecific function calls: we would send a draw message to a chart when we want the chart to draw itself.
- A message is different from a subroutine call, since different objects can respond to the same message in different ways.
- For example, cars, motorcycles, and bicycles will all respond to a stop message, but the actual operations performed are object specific.

Behavior (Object Behavior and methods)

- Methods are similar to functions, procedures, or subroutine in more traditional programming languages, such as COBAL, Basic, or C.
- The area where methods and functions differ, however, is in how they are invoked.
- Figure: Objects respond to messages according to methods defined in its class.



- **Information Hiding (and Encapsulations):-**
- Information hiding is the principle of concealing the internal data and procedures of an object and providing an interface to each object in such a way to reveal as little as possible about its inner workings.
- On object is said to encapsulate the data and a program. This means that user cannot see the inside of the object “capsule,” but can use the object by calling the object’s methods.

Behavior (Object Behavior and methods)

- Encapsulation or information hiding is a design goal of an object-oriented system.
- Rather than allowing an object direct access to another object's data, a message is sent to the target object requesting information.
- Another issue is per-object or per-class protection. In **per-class protection**, the most common form (e.g., Ada, C++, Eiffel), class methods can access any object of that class and not just the receiver. In **per-object protection**, methods can access only the receiver.
- A car engine is an example of encapsulation.

Inheritance (programming by extension)

- Inheritance is a relationship between classes where one class is the parent class of another (derived) class.
- Inheritance allows classes to share and reuse behaviors and attributes.
- The real advantage of inheritance is that we can build upon what we already have and
- Reuse what we already have.

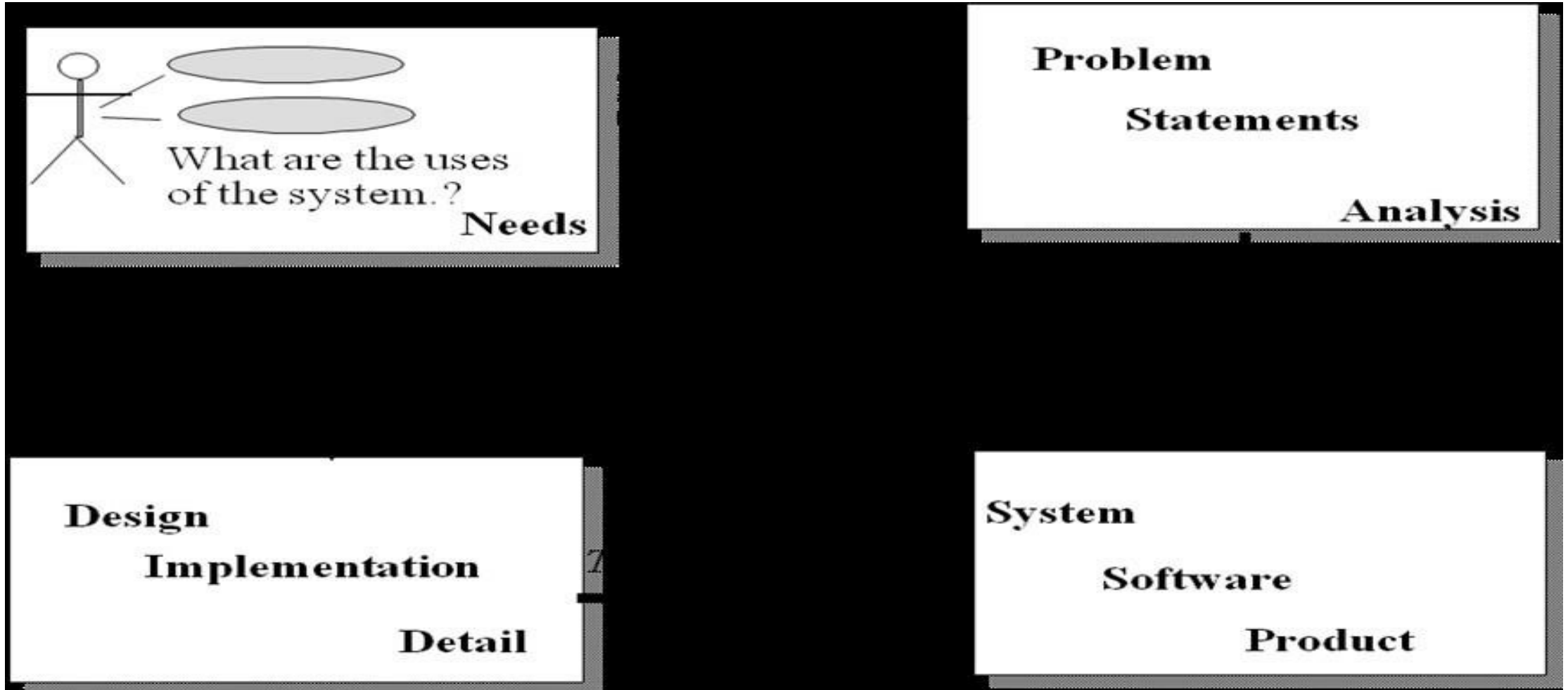
THE SOFTWARE DEVELOPMENT PROCESS

- System development can be viewed as a process. Furthermore, the development itself is a process of change, refinement, transformation, or addition to the existing product.
- Within the process, it is possible to replace one sub process with a new one, as long as the new sub process has the same interface as the old one, to allow it to fit into the process as a whole. With this method of change, it is possible to adapt the new process.
- The process can be divided into small, interacting phases-sub processes. The sub processes must be defined in such a way that they are clearly spelled out, to allow each activity to be performed as independently of other sub processes as possible.

THE SOFTWARE DEVELOPMENT PROCESS

- Each sub process must have the following
 - A description in terms of how it works
 - Specification of the input required for the process
 - Specification of the output to be produced
- The software development process also can be divided into smaller, interacting sub processes. Generally, the software development process can be viewed as a series of transformations, where the output of one transformation becomes the input of the subsequent transformation.

THE SOFTWARE DEVELOPMENT PROCESS



Transformation 1 (analysis)

- translates the users' needs into system requirements and responsibilities.
- The way they use the system can provide insight into the users' requirements.
- For example, one use of the system might be analyzing an incentive payroll system, which will tell us that this capacity must be included in the system requirements.

Transformation 2 (design)

- begins with a problem statement and ends with a detailed design that can be transformed into an operational system.
- This transformation includes the bulk of the software development activity, including the definition of how to build the software, its development, and its testing.
- It also includes the design descriptions, the program, and the testing materials.

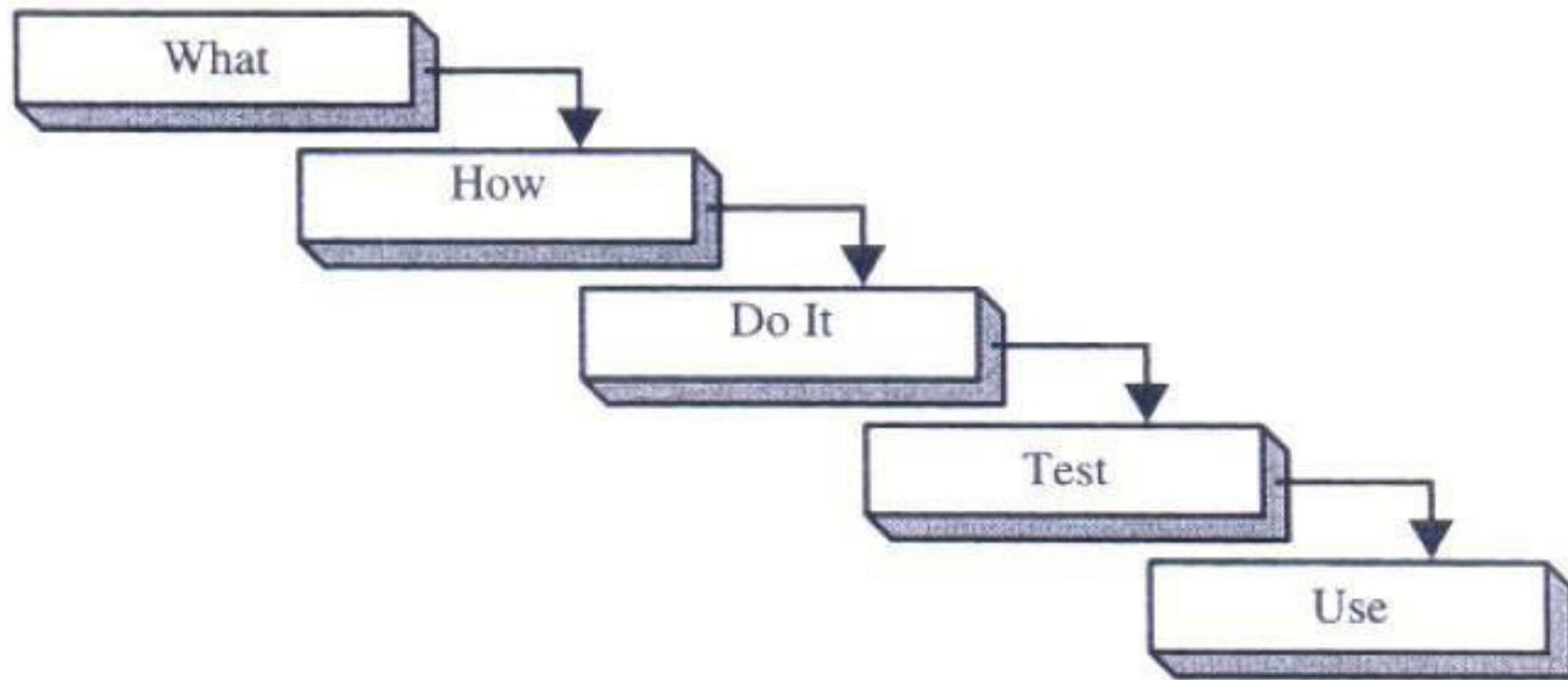
Transformation 3 (implementation)

- refines the detailed design into the system deployment that will satisfy the users' needs.
- This takes into account the equipment, procedures, people, and the like.
- It represents embedding the software product within its operational environment.
- For example, the new compensation method is programmed, new forms are put to use, and new reports now can be printed.

Transformation 3

- An example of the software development process is the ***waterfall approach***, which starts with deciding *what* is to be done (what is the problem).
- Once the requirements have been determined, we next must decide *how* to accomplish them.
- This is followed by a step in which we *do it*, whatever "it" has required us to do. We then must *test* the result to see if we have satisfied the users' requirements.

Transformation 3



Thank You!